Streaming Lakehouse Data Platform

Course: Distributed Systems

Supervisor: PhD. Nguyen Quang Hung

Ngo Quang Hieu – 2311015
Mai Thanh Hoang – 2311048
Le Vu Minh Hieu – 2310953

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City, September 2026

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Table of Contents

1. Introduction
2. Problem & Solution
3. Proposed Solution
4. Conclusion

Motivation & Goals

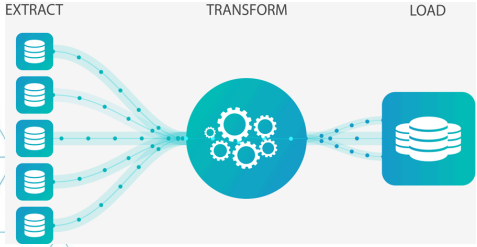
Why a streaming lakehouse

- Eventsim emits a continuous stream of song-play, page-view, and login events onto 3 Kafka topics, not daily batches
- Batch ETL is always some interval behind reality – fine for reports, too slow for "what is happening now"
- Doing this reliably (not just fast) means solving ordering, fault tolerance, exactly-once delivery, and read/write contention on a shared 4-core Spark cluster

Goals

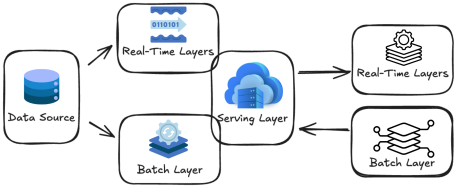
- Build the full Eventsim → Kafka → Spark Structured Streaming → Hudi → dbt → Superset pipeline, end to end
- Apply distributed-systems concepts to a real system: 2-node Kafka replication, checkpointed exactly-once processing, consumer-lag monitoring
- Make the whole system observable (Prometheus/Grafana) and reproducible – single Docker Compose command

The Problem (1/2): Latency and Duplicated Logic



Batch ETL latency

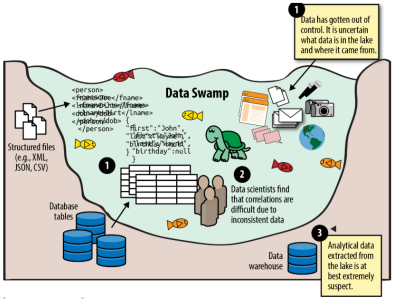
- Data is loaded on a schedule
- Dashboards show the last load, not now



Lambda architecture

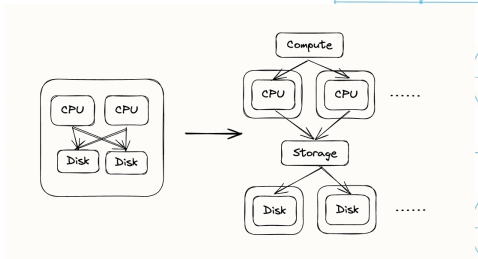
- Batch layer + speed layer
- Same logic written twice, two codebases that can drift apart

The Problem (2/2): The Lake Cannot Serve, Storage Is Coupled



No serving layer on the lake

- Raw files: no row-level update, no ACID
- The lake becomes a data swamp



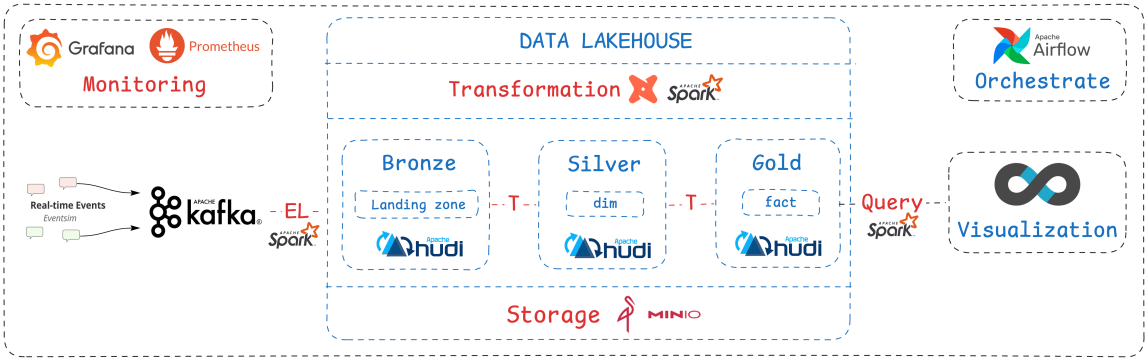
Storage and compute coupled

- Warehouse: scale one, pay for both
- Two copies of data (lake + warehouse)

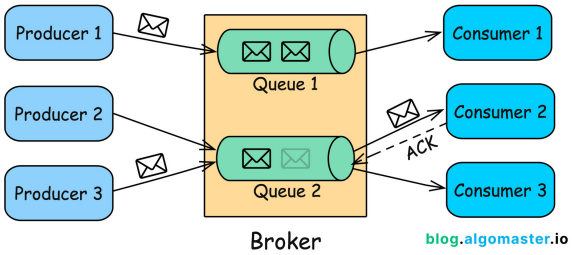
How We Solve It

Problem	Our solution	Tool
Batch ETL latency	Continuous streaming into the lake, no wait for a schedule	Kafka + Spark Streaming
Lambda duplication	One streaming path; dbt reads the same tables, no second speed layer	Spark + dbt
No serving layer	Open table format: ACID, upsert, time travel on object storage	Hudi on MinIO
Coupled storage and compute	Storage (MinIO) and compute (Spark) are separate	MinIO + Spark
Manual, fragile runs	Scheduled DAGs for dbt, compaction, dashboard refresh	Airflow

Streamify: End-to-End Architecture

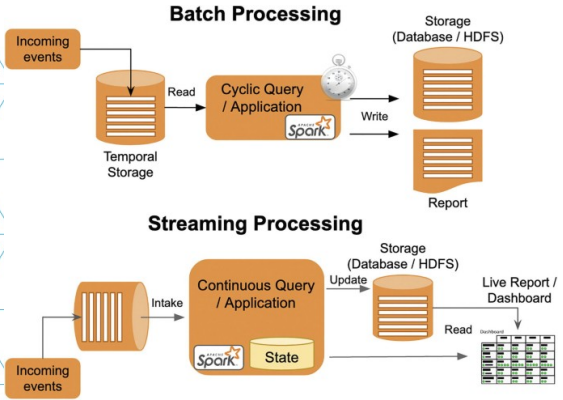


Ingestion Layer: Message Queues & Apache Kafka



- **Message queue (pub-sub)**: producers append to a durable, ordered log; consumers subscribe independently, at their own pace – decoupled in both time and number
- It play a role like a buffer between the producer and consumer, smoothing out bursts of traffic and allowing consumers to catch up
- 2-node KRaft cluster (`broker-1`, `broker-2`) – no ZooKeeper, each node is broker & controller
- 3 topics (`listen_events`, `page_view_events`, `auth_events`), 3 partitions each, replication factor 2 – every partition has a leader on one broker and a follower on the other

Streaming vs. Batch Processing



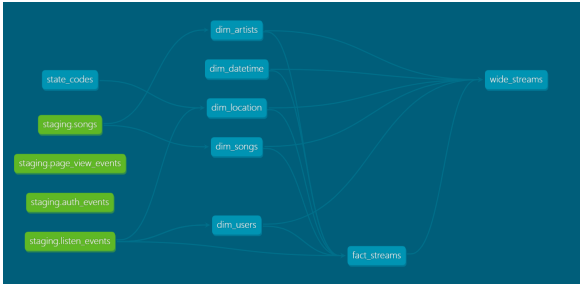
- **Streaming processing** – Spark Structured Streaming reads all Kafka topics continuously, parses and appends straight into storage as bronze (Hudi MERGE_ON_READ)
- Checkpoint volume tracks Kafka offsets and query state – resumes exactly where it left off on restart
- **Batch processing** – Spark also runs as the pipeline's transform layer: dbt reads bronze on a fixed trigger and builds the silver/gold models

Storage Layer & Open Table Formats



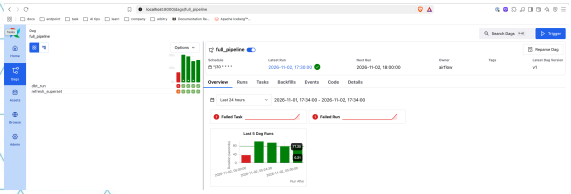
- **Distributed storage** (HDFS, S3, GCS, Azure) → **open file format** (Parquet, ORC) → **open table format** (Hudi, Delta Lake, Iceberg) on top, for ACID upserts and time travel
- Plain files on distributed storage cannot do row-level update/delete or ACID transactions – the open table format adds that, so the lake behaves like a table instead of an unmanaged pile of files
- COPY_ON_WRITE – fast reads, expensive writes; MERGE_ON_READ – cheap streaming appends, merged at read time

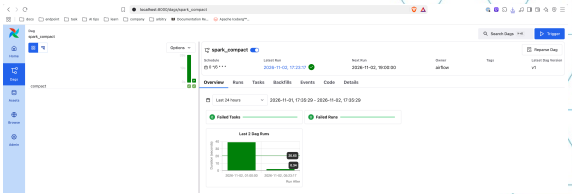
Transformation Layer: dbt



- **Silver** – 5 dimension tables from bronze: `dim_users`, `dim_songs`, `dim_artists`, `dim_locations`, `dim_datetime`
- **Gold** – `fact_streams` (bronze + 5 dims, star schema) and `wide_streams` (`fact_streams` pre-joined with all 5 dims, so Superset needs no dashboard-time joins)
- dbt is a SQL layer over the same Spark cluster: it generates Spark SQL and submits it via the Spark Thrift Server, materializing every model as COPY_ON_WRITE Hudi; fact and big dims update by incremental merge, small dims by overwrite

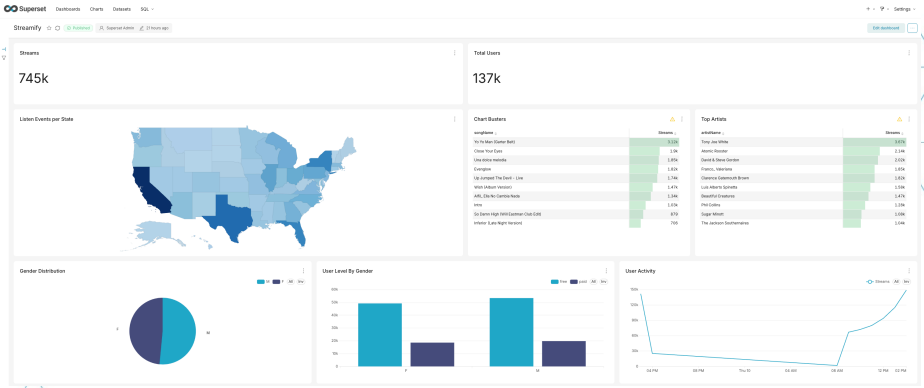
Orchestration Layer: Apache Airflow





- `full_pipeline` (every 30 min): seed, dbt build, then Superset refresh
- `spark_compact` (every 6 h): Hudi compaction; `dbt_adhoc`: run dbt from a form
- One pool slot (`thrift_pool`) so jobs use the Thrift Server one at a time on the 4-core cluster

Serving: Apache Superset



- Superset queries `wide_streams` through the Spark Thrift Server – one flat table, no dashboard-time joins
- **KPIs**: total streams, total users. **Breakdowns**: top songs & artists (Chart Busters, Top Artists), gender distribution, user level by gender
- **Time series**: user activity (stream count per hour). **Geo**: listen events per US state

Monitoring Layer





- **Kafka metrics** (kafka-exporter) – consumer-group lag per partition, messages in/out per second, under-replicated partitions
- **Spark metrics** (Prometheus endpoint) – streaming batch duration & processing rate, executor CPU/memory, active jobs/stages
- **Container metrics** (cAdvisor) – per-container CPU and memory usage across the cluster
- Grafana dashboards make these visible in real time – e.g. rising consumer lag or batch duration flags the pipeline falling behind, not just whether it is running

Conclusion

Results Achieved

- Understood, hands-on, how a streaming platform works end to end: why a Kafka partition needs a follower replica, why the Structured Streaming checkpoint is separate from the consumer-group offset, when MERGE_ON_READ is worth its read-time cost
- Exercised partitioning, replication, checkpointed exactly-once processing, and lag observability against real failure modes, not just in theory
- Confirmed why the streaming-lakehouse pattern closes the batch/lambda-architecture gaps: no duplicated speed-layer logic, a transactionally-updatable lake, storage and compute that scale independently

Limitations

- Small, fixed 4-core Spark cluster, no autoscaling
- Synthetic Eventsim dataset, not validated against real production traffic

Future Directions

- Real or real-scale streaming dataset; autoscaled Kafka/Spark cluster
- Alerting on consumer lag and job failures

- THE END -

Thank you for your attention

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology